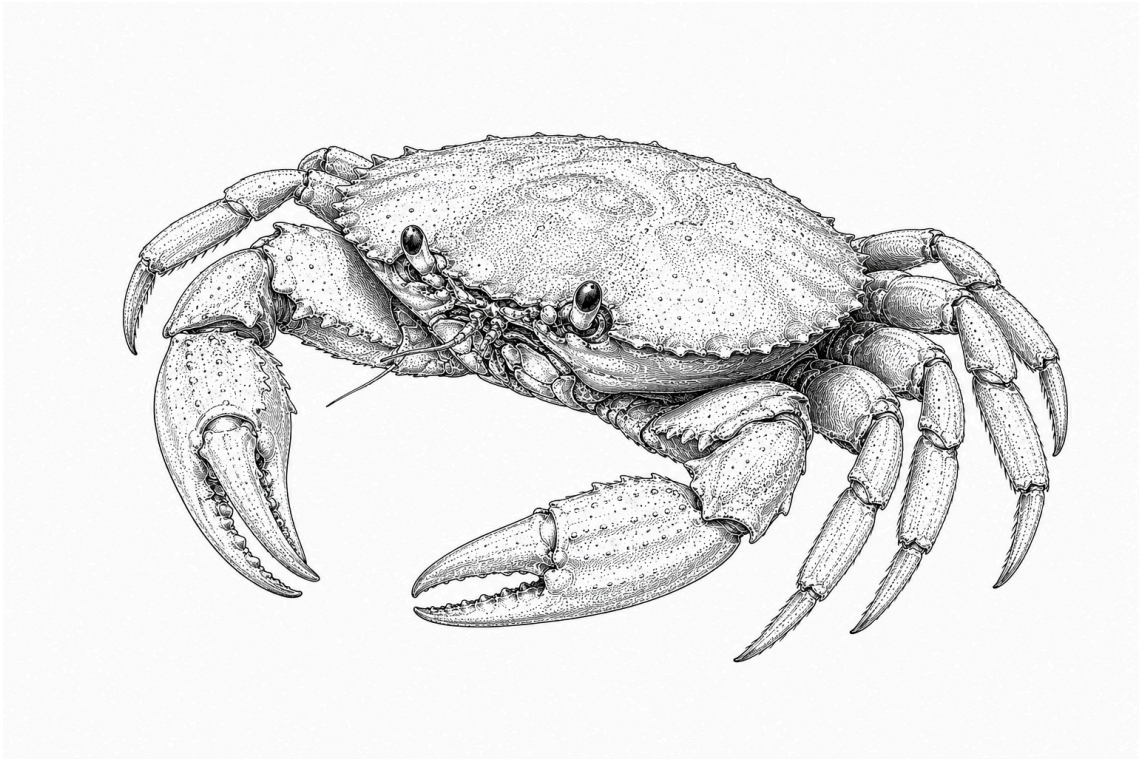
Toolchain Setup

Lab - Advanced Programming



HES-SO Valais//Wallis
Systems Engineering • Infotronics
Advanced Programming • Silvan Zahno
Fall Semester 2026 • I3

Silvan Zahno, **Axel Amand**
07.08.2026 • v1.0 • final

Contents

1 Introduction

In this first laboratory you will set up the software tools used throughout the courses Advanced Programming (AProg), Applied Programming (ProgA) and Model-Driven Software Engineering (MDSE). You will also clone the course repositories and verify that your development environment is working correctly.

At the end of this lab, you will have:

- The Rust toolchain installed (`rustup`, `rustc`, `cargo`)
- The Zed editor installed and configured
- Sublime Merge installed for Git version control
- The `just` task runner installed
- The `langquest` learning tool installed
- SEB installed for exams
- The course repositories cloned locally

1.1 Prerequisites

Before starting, make sure you have:

- Your personal laptop
- Administrator (or sudo) rights
- A working internet connection
- A GitHub account

Complete all installation steps before continuing to the hands-on exercises.

2 Installation

This section guides you through the installation of every tool used in this course. Work through the subsections in order - some tools depend on others being installed first.

2.1 Zed Code Editor

Zed is a high-performance, multiplayer code editor built in Rust. It is the primary editor used throughout this course.

Figure 1 - Zed - high-performance code editor

2.1.1 Installation

Visit zed.dev and download the installer for your platform.

Download the **`.dmg`**, open it and drag *Zed* to the *Applications* folder.

Install the Zed CLI tool by running the following command in Zed Press **`cmd-shift-p`** and type **`cli:install cli binary`**

Open a terminal and run the official installer script:

```
curl https://zed.dev/install.sh | sh
```

Download and run the **`.exe`** installer
Add **`zed.exe`** to your **`PATH`** if the installer does not do it automatically.

2.1.2 Verification

Open Zed and confirm the welcome screen appears. Sign in with your GitHub account to enable Artificial Intelligence (AI) features.

2.2 Rust Toolchain

Rust is installed and managed through **`rustup`** - the official Rust toolchain installer. **`rustup`** handles installation of the compiler (**`rustc`**), the package manager and build tool (**`cargo`**), and additional targets or components.

Figure 2 - Rust Programming Language

2.2.1 Installation

Visit rust-lang.org/tools/install for full instructions.

Open a terminal and run:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Follow the on-screen prompts (the default installation is recommended). After installation, **restart your terminal**.

Download and run **`rustup-init.exe`** from rustup.rs. Install the *MSVC* build tools when prompted.

2.2.2 Install `cargo` addons

In a new terminal, run the following commands to install useful **`cargo`** addons for code formatting and linting.

```
# Install rustfmt for code formatting
rustup component add rustfmt
# Install clippy for linting and code analysis
rustup component add clippy
# Install cargo-edit and cargo-generate for managing dependencies and project templates
cargo install cargo-edit cargo-generate
```

2.2.3 Verification

Verify that the following command prints a version number without error.

```
rustup --version
rustc --version
cargo --version
cargo fmt --version
cargo clippy --version
cargo generate --version
```

If they fail, ensure **`$HOME/.cargo/bin`** (macOS/Linux) or **`%USERPROFILE%\.cargo\bin`** (Windows) is on your **`PATH`**.

2.3 Just - Command Runner

`just` is a handy command runner (similar to **`make`**) that reads a **`justfile`** in your project. It is distributed as a binary Rust crate and installed via **`cargo`**.

2.3.1 Installation

```
cargo install just
```

This may take a few minutes on the first install as it downloads and compiles the crate and its dependencies.

2.3.2 Verification

Verify that the following command prints a version number without error.

```
just --version
```

2.4 Langquest

`LangQuest` is a terminal-based vocabulary quiz tool used to practice programming terminology and concepts. It is distributed as binary under LangQuest on Github.

Figure 3 - Langquest - vocabulary quiz tool

2.4.1 Installation

```
curl -fsSL https://raw.githubusercontent.com/tschinz/langquest/
refs/heads/main/scripts/install_latest_release.sh | sh
```

This installs LangQuest under **`$HOME/.local/bin`**.

```
irm https://raw.githubusercontent.com/tschinz/langquest/refs/heads/
main/scripts/install_latest_release.ps1 | iex
```

This installs LangQuest under **`%LOCALAPPDATA%\Programs\lq\bin\lq.exe`**.

2.4.2 Verification

Verify that the following command prints a version number without error:

```
lq -k
```

Check that the keys correspond to the followings:

```
progress_key_sha256:
869e730b3d6be463c5474a41f802711943e330827a7c4ddd6bcdf2dc9f4abc3a
attest_key_sha256:
af30b6a2512c11e8015da1522b55d61926f96591c3193a63f2d1b64f6f53b5b7
solution_key_sha256:
1d142c246028f02e928682c88b3280412d4ff82b0cbc275db68a236947c89283
```

Define default editor for Markdown files (**`*.md`**) and Rust Files (**`*.rs`**) to Zed. This will allow you to open files form lanquest with the shortcut **`e`**.

2.5 GitHub CLI (gh)

The GitHub CLI (**gh**) brings GitHub to your terminal. It lets you authenticate Git operations and interact with repositories, issues and pull requests without leaving the command line.

Figure 4 - GitHub CLI - GitHub in your terminal

2.5.1 Installation

Visit cli.github.com for full instructions.

Install via Homebrew. If Homebrew is not yet installed, install it first:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/
Homebrew/install/HEAD/install.sh)"
```

Then install the GitHub CLI:

```
brew install gh
```

Install from the official APT repository:

```
sudo apt-get update
sudo apt-get install -y wget

# Add GitHub CLI's official package repository
sudo mkdir -p -m 755 /etc/apt/keyrings
wget -qO- https://cli.github.com/packages/githubcli-archive-keyring.gpg \
  | sudo tee /etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null
sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/githubcli-archive-
keyring.gpg] https://cli.github.com/packages stable main" \
  | sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null

# Install the GitHub CLI
sudo apt-get update
sudo apt-get install -y gh
```

Download and run the installer from cli.github.com, or install via **winget**:

```
winget install --id GitHub.cli
```

Note: You will need a GitHub account to authenticate. If you do not have one, create an account at github.com/signup before proceeding.

2.5.2 Authentication

Once installed, authenticate the GitHub CLI with your GitHub account:

```
gh auth login   # authenticate the GitHub CLI (opens a browser)
```

Follow the interactive prompts: choose **`GitHub.com`**, then **`HTTPS`** (or **`SSH`** if you already have a key configured and are familiar with), and log in via your web browser when prompted.

2.5.3 GitHub Student Extension

This course also uses the **`gh-student`** extension. Install it with:

```
gh extension install foundation50/gh-student
```

2.5.4 Verification

Verify that the following commands succeed without error.

```
gh --version
gh auth status
gh extension list
```

2.6 Sublime Merge

Sublime Merge is a fast, feature-rich Git client with a clean visual interface. It is used for reviewing diffs, staging individual hunks and managing branches.

Figure 5 - SublimeMerge - a git GUI client

2.6.1 Installation

Visit sublimemerge.com and download the installer for your platform.

Open the downloaded **`.dmg`** and drag *Sublime Merge* to the *Applications* folder.

Follow the package manager instructions at sublimemerge.com/docs/linux_repositories.

Run the **`.exe`** installer.

2.6.2 Verification

Open Sublime Merge. It should display a welcome screen offering to open or clone a repository.

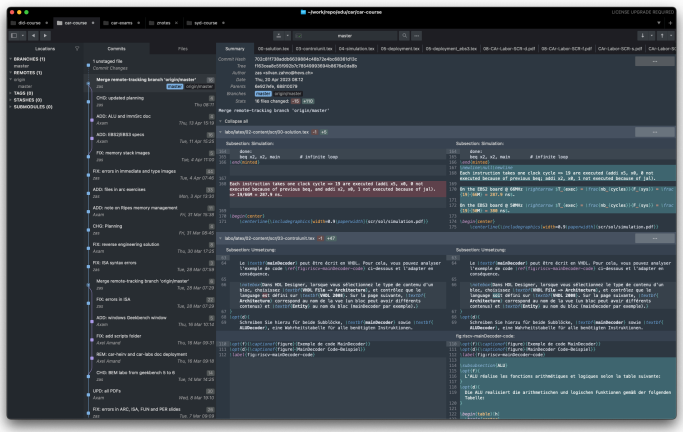


Figure 6 - Sublime Merge - visual Git client

2.7 Safe Exam Browser (SEB)

The Safe Exam Browser is used during in-class written exams. It locks the workstation to an exam environment, preventing access to unauthorized resources.

Figure 7 - Safe Exam Browser - secured exam environment

2.7.1 Installation

Visit safeexambrowser.org/download_en.html and download the version for your operating system.

Open the `.dmg` and follow the on-screen instructions. You may need to grant accessibility permissions in *System Settings* > *Privacy & Security*.

Run the `.msi` installer.

Important: Install SEB *before* the first graded exam. Test the launch at least once to ensure it works on your machine. Last-minute installation issues are not grounds for extra time during an exam.

2.7.2 Verification

Launch SEB. It should start and display its initial configuration screen. You can close it immediately - you do not need an exam configuration file to test the launch.

2.8 Get the Course Repositories

The lab and project materials are hosted in GitHub repositories. Via a classroom link these repositories will be forked, you will then work on your own fork of the repository. The repository (**`aprog-labs`**) hold all the file for the exercise session as well as lab sessions.

1. Send your **full name**, **`@hevs.ch` email address** and **GitHub username** to the course instructor via Teams.
2. Use the classroom link to accept the invitation to the assignment and join the organization.
3. Your repository of the assignment wil be created in the organization hei-synd-swd-stud.
4. Clone your forked repository to your local machine using the commandline or SublimeMerge (**`File`** ⇨ **`Clone Repository`**):

```
mkdir -p ~/work                     # Create work directory if it doesn't exist
cd ~/work/                          # Goto work directory
git clone repository-url aprog-labs # Clone the repository replace with actual URL
cd ~/work/aprog-labs                # Change to the cloned repository directory
zed .                               # Open the project folder in Zed
```

```
mkdir "%USERPROFILE%\work"            # Create work directory if it doesn't exist
cd "%USERPROFILE%\work"               # Goto work directory
git clone repository-url aprog-labs  # Clone the repository, replace with actual URL
cd "%USERPROFILE%\work\aprog-labs"    # Change to the cloned repository directory
zed.exe .                             # Open the project folder in Zed
```

The repositories are part of your grade. You must properly fork, commit and tag your work weekly to receive points. Follow the instructions carefully and ask for help if you encounter issues with Git, Sublime Merge or GitHub.

3 Hands-on

This section introduces the installed tools through practical exercises. Work through each subsection in order and record your observations.

3.1 Exercise 1 - Rust Toolchain

The Rust ecosystem is built around three command-line tools that you will use every day:

Tool	Role
`rustup`	Toolchain manager - install, update and switch Rust versions and components
`rustc`	Compiler - transforms **`.rs`** source files into executables or libraries
`cargo`	Build system and package manager - create projects, manage dependencies, run tests

Table 1 - The three core Rust toolchain components

3.1.1 Exercise 1.1 - Explore `rustup`

Run each command below and note the output.

```
# Show installed toolchains and active toolchain
$ rustup show

# List components currently installed (e.g., rustfmt, clippy)
$ rustup component list --installed

# Update to the latest stable toolchain
$ rustup update
```

3.1.2 Exercise 1.2 - Hello World with `rustc`

Compile and run a Rust program *without Cargo* to understand what the compiler does directly.

Have a look at the **`hello.rs`** file in the lab repository under **`aprog-labs/labs/01-tool/01-rustc-hello`** or create a new one with the following content:

```
fn main() {
    println!("Hello, Toolchain!");
}
```

```
cd ~/work/aprog-labs/labs/01-tool/01-rustc-hello
rustc hello.rs -o hello  # Compiles hello.rs with rustc
./hello                  # Executes the compiled binary
```

```
cd /d "%USERPROFILE%\work\aprog-labs\labs\01-tool\01-rustc-hello"
rustc hello.rs -o hello.exe  # Compiles hello.rs with rustc
.\hello.exe                  # Executes the compiled binary
```

Note: Direct **`rustc`** invocation is rarely used in practice. Cargo handles compilation, dependency resolution and linking automatically. This exercise shows what happens under the hood.

3.1.3 Exercise 1.3 - Create a `cargo` Project

Create and run a new Rust project using **`cargo`**. **`cargo`** manages the entire build process, including compiling your code, downloading and building dependencies, and running tests.

```
cd ~/work/aprog-labs/labs/01-tool/
cargo new cargo-hello      # create a new binary crate
cd cargo-hello             # change to the new project directory
zed .                      # Open the project folder in Zed
```

```
cd /d "%USERPROFILE%\aprog-labs\labs\01-tool\ " # Goto work
directory
cargo new cargo-hello          # create a new binary crate
cd cargo-hello                 # change to the new project directory
zed.exe .                      # Open the project folder in Zed
```

In the Zed Terminal (View ⇨ Terminal Panel), run the following commands:

```
# Build and run in one step
$ cargo run

# Build only (produces the binary in target/debug/)
$ cargo build

# Build in release mode (optimizations enabled)
# The binary will be in target/release/ and will run faster than the debug build.
$ cargo build --release
```

Open **`Cargo.toml`** in Zed and inspect the **`[package]`** and **`[dependencies]`** sections.

3.2 Exercise 2 - Hello World

Here we will work with a simple "Hello, World!" project which is in **`aprog-labs/labs/01-tool/02-hello-world`**. This project contains a **`justfile`** with pre-defined tasks and serves as a playground for exploring **`just`**, **`cargo`** and **`rustdoc`** .

```
cd ~/work/aprog-labs/labs/01-tool/02-hello-world
zed .
```

```
cd /d "%USERPROFILE%\aprog-labs\labs\01-tool\02-hello-world"
zed.exe .
```

3.2.1 Exercise 2.1 - Execute a Just Script

`just` reads *recipes* from a **`justfile`** (similar to a **`Makefile`**). In this course, lab repositories ship with a **`justfile`** containing pre-defined development tasks.

Have a look at the **`justfile`** in the project root **`aprog-labs/labs/01-tool/02-hello-world`**. It defines several recipes for building, running, documenting and cleaning the project.

```
$ just
Available recipes:
    build     # Build the project in debug mode
    check     # Run cargo check (fast compile check, no codegen)
    clean     # Clean build artifacts
    clippy    # Run clippy with strict warnings
    default   # List all available commands
    doc       # Generate and open rustdoc documentation
    doc-build # Generate rustdoc documentation without opening
    info      # Print environment info (OS, arch, toolchains)
    run       # Run the project
    rustfmt   # Format source with rustfmt
    test      # Run all tests
```

Run the different recipes (e.g., **`just run`**, **`just doc`**, **`just clean`**) and observe the output. Note how **`just`** abstracts away the underlying commands.

3.2.2 Exercise 2.2 - Generate Rustdoc

Rust has first-class documentation support built into the language. Documentation comments are written with `///` (item-level) or `//!` (module-level) and support Markdown formatting.

Read the rustdoc comments in **`aprog-labs/labs/01-tool/02-hello-world/src/lib.rs`**.

Generate the documentation with:

```
just doc
```

Navigate the generated documentation in your browser and compare them with the project files **`02-hello-world/src/lib.rs`** and **`02-hello-world/src/main.rs`**.

3.3 Exercise 3 - First Experience with Langquest `lq`

`langquest` is a vocabulary training tool. Exercises in **`aprog-labs/exercises`** are designed to be used with **`langquest`** to reinforce key concepts and terminology from the course.

```
zed ~/work/aprog-labs\exercises

# inside the zed terminal, run:
lq --repo ~/work/aprog-labs/exercises
```

```
zed.exe /d "%USERPROFILE%\work\aprog-labs\exercises"

# inside the zed terminal, run:
lq --repo ~/work/aprog-labs/exercises
```

Solve the first exercise.

4 Wrap-up

Now that the hands-on exercises are complete, you will commit your work using the **Conventional Commits** specification and mark the milestone with a Git tag.

4.1 Exercise 4 - Commit your work

4.1.1 Conventional Commits

Conventional Commits is a lightweight convention for writing structured, machine-readable commit messages. It makes the project history easy to navigate for humans and enables automatic changelog generation and semantic versioning.

4.1.2 Message Format

```
<type>(<scope>): <short summary>
│       │              │
│       │              └── Imperative mood, lowercase, no period at end
│       └── Optional: the part of the project affected (file, module, component)
└── Required: feat | fix | docs | style | refactor | test | chore | perf | revert
```

Type	When to use	Example
`feat`	A new feature or capability	`feat(hello): add greeting with user name`
`fix`	A bug fix	`fix(cli): prevent crash when no argument is provided`
`docs`	Documentation-only changes	`docs(readme): add Cargo installation instructions`
`style`	Formatting, whitespace, or code style changes with no behavior change	`style: format project with cargo fmt`
`refactor`	Code restructuring without changing behavior	`refactor(main): move greeting logic into separate function`
`test`	Adding or correcting tests	`test(hello): add test for empty user name`
`chore`	Project maintenance, tooling, or configuration changes	`chore: add .gitignore for Rust projects`
`perf`	Performance improvement without changing functionality	`perf(hello): avoid repeated string allocations`
`revert`	Undo a previous commit	`revert: feat(hello): add greeting with user name`

Table 2 - Conventional Commits types and usage examples

Each commit should focus on a single logical change. If you find yourself staging unrelated files, consider splitting the change into multiple commits for clarity.

4.1.3 Git Stages

Before creating commits, it is important to understand Git's three stages.

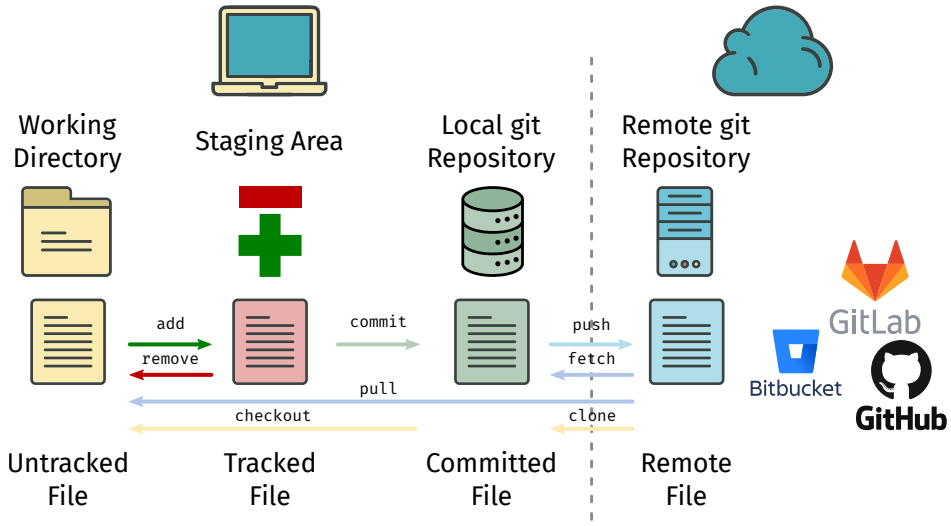


Figure 8 - The three Git stages: working directory, staging area and repository

A file moves through three states during the Git workflow:

1. **Working Directory** - Files you are currently editing. Changes exist only on your computer and are not yet part of the project history.
2. **Staging Area** - Files selected for inclusion in the next commit. Use **`git add`** to move changes from the working directory to the staging area.
3. **Repository** - The permanent project history. When you run **`git commit`**, all staged changes are recorded in the repository as a new commit.

Staging allows you to choose exactly which changes belong to a commit. This helps create small, focused commits that are easier to understand, review, and revert.

4.1.4 Exercise 4.1 - Make Conventional Commits

At this point you have created several projects and files during the lab. Your goal is to organize these changes into a small number of meaningful commits.

Before committing, inspect the repository status in SublimeMerge or via commandline to see all modified and untracked files:

```
1 git status
```

Review the modified and untracked files and decide which files belong together logically.

A good commit should:

- Represent a single logical change.
- Have a clear Conventional Commit message.
- Contain only related files.

- Be understandable without additional explanation.

For example:

- Adding the **`01-rustc-hello`** project could be one **`feat`** commit.
- Adding the cargo-based **`hello-world`** project could be another **`feat`** commit.
- Adding configuration files, templates, or helper resources could be a **`chore`** commit.

Stage only the relevant files for each commit. Use git add from the command line or stage individual files in Sublime Merge.

The process for each commit is:

1. Review the changed files.
2. Stage only the files related to one logical change.
3. Write a Conventional Commit message.
4. Create the commit.
5. Repeat for the next logical change.
6. Push the commits to GitHub.

The following screenshots illustrate the workflow in Sublime Merge:

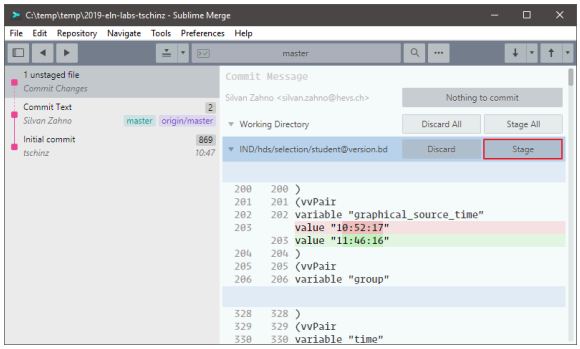


Figure 9 - Step 1 Stage only the files related to the change

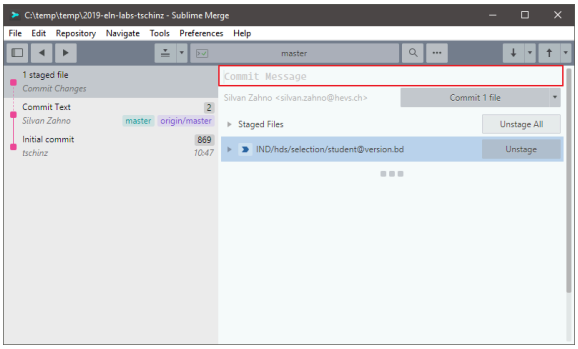


Figure 10 - Step 2 Write a clear, conventional commit message

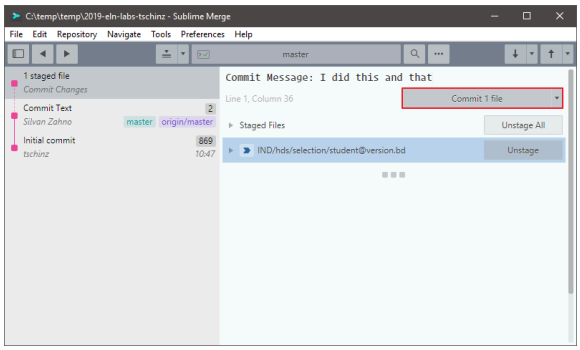


Figure 11 - Step 3 Commit the change with the descriptive message

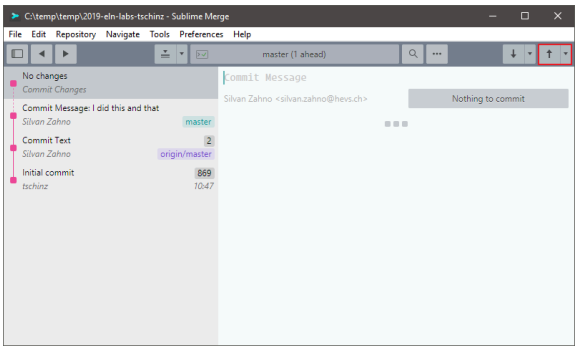


Figure 12 - Step 4 Push the commit to the remote repository

4.1.5 Exercise 4.2 - Git Tag

A Git tag is a named reference to a specific commit.

Unlike branches, tags do not move. They are commonly used to mark:

- Releases (v1.0.0)

- Project milestones
- Assignment submissions

In this course, a tag identifies the exact version of your work that will be evaluated. Creating a tag ensures that the grader can always retrieve the correct submission, even if additional commits are made later.

After creating your commits, create a tag named **`01-tool`** on the latest commit and push it to GitHub.

SublimeMerge Right Click on the commit ⇨ **`Create`** ⇨ **`Create Tag at Commit`** and enter the tag name **`01-tool`** with a descriptive message.

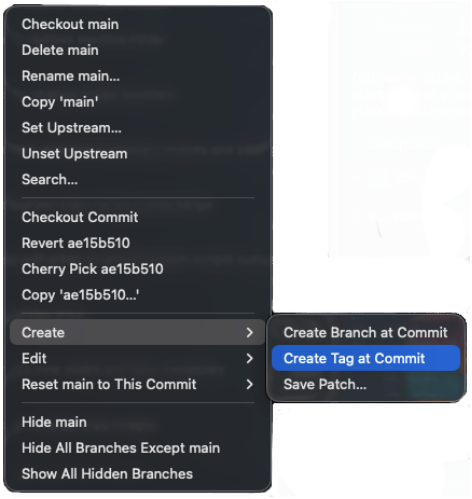


Figure 13 - Create a tag using SublimeMerge.

Do Not Forget to Push the Tag

Commandline

Alternatively, you can create and push the tag from the command line:

```
$ git tag -a 01-tool -m "feat: complete lab 01-tool"
$ git push --tags
```

Have a look on github and verify if all commits and tags are pushed.

5 Conclusion

In this laboratory you set up a complete Rust development environment and took your first steps with each of its components.

5.1 Check List

Before you leave, confirm that every item below is complete:

Installation:

- ☐ Zed opens and the welcome screen is visible
- ☐ **`rustup --version`**, **`rustc --version`** and **`cargo --version`** all return valid versions
- ☐ **`just --version`** returns a valid version
- ☐ **`langquest --help`** displays the help message
- ☐ Sublime Merge opens and can list repositories
- ☐ Safe Exam Browser launches without error
- ☐ **`aprog-labs`** repository is cloned and accessible locally

Hands-on Exercises:

- ☐ **`01-rustc-hello`** is compiled.
- ☐ **`cargo-hello`** project is created.
- ☐ **`02-hello-world`** was explored using the **`justfile`** recipes
- ☐ The first exercise was solved in langquest **`lq`**

Wrap-up:

- ☐ Changes in the **`aprog-labs`** repository has been commited and pushed
- ☐ Commit messages follow the **`<type>(<scope>): <summary>`** format
- ☐ Tag **`01-tool`** is created and pushed

Congratulations! Your development environment is fully operational. You are ready to start writing Rust code in the next laboratory sessions.

Glossary

***AProg* – Advanced Programming**: Course covering advanced programming concepts using Rust. 3

***MDSE* – Model-Driven Software Engineering**: Course focused on software engineering practices that use models to drive development. 3

***ProgA* – Applied Programming**: Project-oriented course focused on applying programming concepts in practice. 3

***AI* – Artificial Intelligence**: Field of computer science focused on creating systems capable of performing tasks that typically require human intelligence. 4

***SEB* – Safe Exam Browser**: Secure web browser environment used for in-class written exams. 2, 3, 11, 11, 11